



STM-Multifrontal QR: Streaming Task Mapping Multifrontal QR Factorization Empowered by GCN

Shengle Lin
lsl036@hnu.edu.cn
Hunan University
Changsha, China

Wangdong Yang*
yangwangdong@hnu.edu.cn
Hunan University
Changsha, China

Haotian Wang
wanghaotian@hnu.edu.cn
Hunan University
Changsha, China

Qinyun Tsai
hnutsai@hnu.edu.cn
Hunan University
Changsha, China

Kenli Li
lkl@hnu.edu.cn
Hunan University
Changsha, China

ABSTRACT

Multifrontal QR algorithm, which consists of symbolic analysis and numerical factorization, is a high-performance algorithm for orthogonal factorizing sparse matrix. In this work, a graph convolutional network (GCN) for adaptively selecting the optimal reordering algorithm is proposed in symbolic analysis. Using our GCN adaptive classifier, the average numerical factorization time is reduced by 20.78% compared with the default approach, and the additional memory overhead is approximately 4% higher than that of prior work. Moreover, for numerical factorization, an optimized tasks stream parallel processing strategy is proposed and a more efficient computing task mapping framework for NUMA architecture is adopted in this paper, which called STM-Multifrontal QR factorization. Numerical experiments on the TaiShan Server show average 1.22x performance gains over the original SuiteSparseQR. Nearly 80% of datasets have achieved better performance compared with the MKL sparse QR on Intel Xeon 6248.

CCS CONCEPTS

• **Mathematics of computing** → **Mathematical software performance**; • **Computing methodologies** → Symbolic calculus algorithms; Neural networks; Algebraic algorithms; Shared memory algorithms.

KEYWORDS

Multifrontal QR Factorization, Graph Convolutional Network, Task Stream Processing, Task Mapping Optimization, NUMA Multicore Architecture

*Corresponding authors

ACM Reference Format:

Shengle Lin, Wangdong Yang, Haotian Wang, Qinyun Tsai, and Kenli Li. 2021. STM-Multifrontal QR: Streaming Task Mapping Multifrontal QR Factorization Empowered by GCN. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, November 14–19, 2021, St. Louis, MO, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3458817.3476199>

1 INTRODUCTION

QR factorization is one of the critical kernels in the modern computational science. It is usually applied to least squares problems ($\min_x \|b - Ax\|_2$), eigenvalue calculation problems, directly finding the basic solution to linear systems $Ax = b$, preconditioners calculation of the iterative solver and some other matrix numerical computation problems[22, 23]. The increasing complexity of application scenarios makes it necessary to effectively tackle the case of a sparse matrix, whose decomposition performance is strongly affected by the fill-in, i.e., the new nonzero entries introduced during factorization.

Several algorithms have been proposed to reach theoretical peak computational performance when factorizing a large sparse system. Early sparse QR factorization algorithms based on Givens rotations operate on sparse matrix A one row or one column at a time [17, 18, 26], where the fill-in is minimized by exploiting the sparsity of the input matrix. These earliest methods can not achieve a satisfactory performance due to the irregular access of memory imposed by the data structures that represent a sparse matrix. The *multifrontal method* is first developed for the factorization of sparse, indefinite, symmetric matrices by Liu [27, 28], and is then utilized in QR factorization. In multifrontal algorithm, the factorization of a large sparse matrix is performed in a sequence of relatively small dense sub-matrices (called *frontal matrices* or *fronts*) that can better exploit the memory subsystems and easily be handled in parallel [13, 31].

There are several parallel implementations of multifrontal QR methods for shared-memory and heterogeneous system [2, 29]. For instance, Davis extended the parallelism of this algorithm by exploiting multicore architecture [11], and Buttari[5] further proposed a fine-grained partitioning of tasks based on [11]. Yeralan further presented a GPU-aware algorithm for multifrontal sparse QR factorization by extending a communication-avoiding strategy[37]. The above-mentioned schemes usually consist two independent steps: symbolic analysis and numerical factorization. Unfortunately,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SC '21, November 14–19, 2021, St. Louis, MO, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8442-1/21/11...\$15.00

<https://doi.org/10.1145/3458817.3476199>

they all suffer from the reordering restrictions and lack of affinity on Non-Uniform Memory Access (NUMA) architecture.

Within this context, this paper optimizes multifrontal sparse QR factorization based on the SuiteSparseQR [15]. In the symbolic analysis phase, our work equips with the graph convolutional network (GCN) to empower a self-adapting selection from several current reordering methods. The result shows that by using the adaptive classifier, the numerical factorization efficiency can be improved by up to 1.2x compared with the default method of SuiteSparseQR with only 3.5% extra storage overhead. Moreover, the task mapping framework with stream processing strategy boosts parallelism during factorization and NUMA node affinity optimization increases the factorization efficiency of each thread on NUMA architecture. Collectively, the above mentioned optimization measures are referred to as *streaming task mapping* (STM) multifrontal QR. The testing results based on 1864 square matrices of the University of Florida Sparse Matrix Collection (UF dataset) show that our streaming task process strategy has achieves a 5 ~ 10% speedup, and the performance is further improved by nearly 20% after implementing the task mapping technique. On nearly 80% of the datasets, STM-Multifrontal QR can perform the same or even better compared with MKL sparse QR. Combined together, our three unique optimization techniques manifest substantial performance gains over the original SuiteSparseQR (an average 1.22x speedup).

2 BACKGROUND AND MOTIVATION

2.1 Reordering for fill-in reduction

When solving a sparse equation set $Ax = b$ or solving a least-square problem, the first step is to reorder the rows and columns of the sparse matrix A so the resulting factors produce fewer fill-in elements, that is, the elements are zero initially but, during the numerical factorization become nonzero. Similarly, the goal of the pre-ordering of multifrontal QR is to find a column permutation P so that $(AP)^T AP = LL^T$ produces fewer fill-in entries than the factors L of the original matrix $A^T A$. It is equivalent to a node elimination process in the undirected graph corresponding to the sparse matrix $A^T A$.

Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ be an undirected graph with a vertex set $\mathcal{V}$ containing $n = |\mathcal{V}|$ nodes (matrix size), edge set $\mathcal{E}$ with $m = |\mathcal{E}|$ edges (nonzeros). Each edge $(u, v) \in \mathcal{E}$ is a pair of vertices of $\mathcal{V}$, which means entries (u, v) and (v, u) of symmetric matrix A are nonzero. An elimination graph $\mathcal{G}_k(A) = \{\mathcal{V}_k, \mathcal{E}_k\}$ is composed of a vertex set $\mathcal{V}_k = \{k+1, \dots, n\}$ ($0 < k < n$) and an edge set, where

$$\mathcal{E}_k = \mathcal{E}_{k-1} \cup \{(u, v) : (u, k), (k, v) \in \mathcal{E}_{k-1}\} \setminus \{(k, t) \in \mathcal{E}_{k-1}\}. \quad (1)$$

It means edges are added to $\mathcal{E}_{k-1}$ to make the nodes adjacent to k -th pivot in $\mathcal{G}_{k-1}$ a *clique*. With these definitions, the fill-in entries and the elimination dependencies between nodes of a sparse symmetric matrix $A^T A$ can be reflected to the corresponding undirected elimination graph $\mathcal{G}_k(A)$.

For instance, a 6×6 symmetric matrix A' and its undirected graph with 3 steps elimination is shown to the left in Fig. 1. The number index of nodes in $\mathcal{G}$ also indicates the order of elimination. Since the decomposition of node 1 will affect the graph structure of node 2, the calculation of node 2 must wait until node 1 completes. After node 1 elimination, node 2 and node 4 will form a new edge

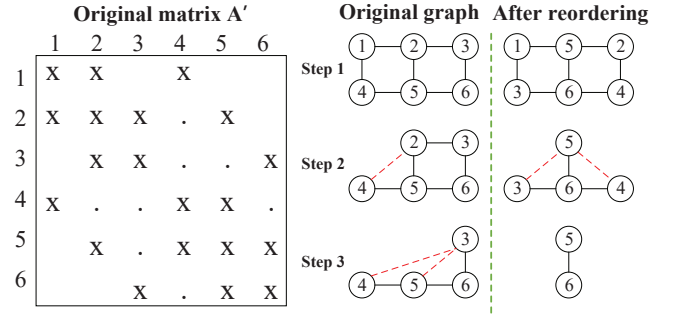


Figure 1: A sample 6×6 symmetric matrix $A' = A^T A$, the corresponding elimination graph (left) and the elimination graph after best reordering (right).

shown with red dash lines because of $(2, 1), (1, 4) \in \mathcal{E}_0$, that is equivalent to the fill-in elements of $A'_{2,4}$ and $A'_{4,2}$. Note that the fill-in amount is appended after each elimination step because new edges are created. The best-reordered result is shown to the right in Fig. 1. In this undirected graph corresponding to the reordered matrix, nodes 1, 2 (then 3, 4) can be eliminated in parallel, with only 4 fill-ins in the matrix.

In summary, finding an optimal reordering strategy is a huge performance boost for sparse QR factorization, but this problem is NP-complete[36]. In order to obtain a better nonzero pattern of the sparse matrix, abundant heuristic algorithms have been proposed: (1) ordering the nodes with approximate minimum degree first, such as approximate minimum degree (AMD)[1] and column-AMD (COLAMD)[12], (2) nested dissection that partitions the graph with node separators, e.g. METIS and NESDIS[6].

2.2 Construct elimination tree and supernodes

The second step of symbolic analysis is to obtain a column elimination tree that can describe the nonzero pattern of the matrix A by performing a Cholesky symbolic factorization of $A^T A$ [21]. The entire analysis proceeds with the Cholesky factor L^T , because the pattern of L is an upper bound on the pattern of QR factor R [8]. Each node on this elimination tree represents one corresponding column of matrix A . The rules for constructing elimination tree are as follows: node j is the parent of node i if $j > i$ and j is the smallest index for which $r_{i,j} \neq 0$.

In the next step, a *supernode* tree is formed from the column elimination tree. A *supernode* of factor R represents a battery of rows with the identical structure of nonzeros. In general, row i and $i+1$ are merged into a *relaxed supernode* if the patterns of $R_{i,i+1:n}$ and $R_{i+1,i+1:n}$ are similar (not necessarily identical)[4]. Each node in supernode tree gives rise to a corresponding *frontal matrix*, which is a dense submatrix contained a subset of the rows of the sparse matrix A . During the numerical factorization phase, QR factorization is performed on these frontal matrices instead of individual vertices, thereby leading to a blocked parallel algorithm. Fig. 2 shows an example of sparse matrix A , factor R , and supernode tree. In the elimination tree shown at the bottom left of the figure, the supernodes corresponding to the division in R are surrounded by a dotted box.

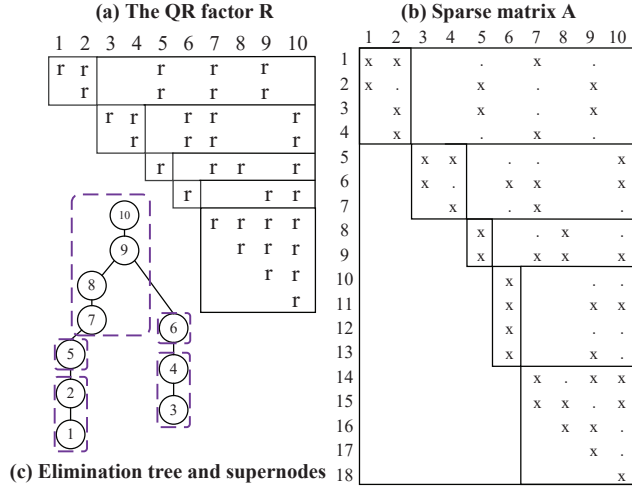


Figure 2: (a) QR factor R, (b) sparse matrix A and (c) corresponding trees (elimination tree and the division of supernodes) of them. Each 'X' represents a nonzero in matrix. Fill-in element is shown as a dot(.).

2.3 Supernodes assignment and parallelism

After generating supernodes, it is necessary to further assign them into different tasks so that the factorizing process can be performed in parallel by multiple threads. The work of each dense frontal matrix f is defined as floating-point work for the subtree rooted at f in supernode tree. Node f is a big node if the total work of its subtree is greater than $\max(\omega/\alpha, \beta)$, where ω is floating-point count for the dense QR factorization, α is recommended to be twice the number of cores and β is user-defined parameter that controls the minimum computation for each thread. This process[11] keeps the floating-point computation for each task as close as possible. An example of the assignment is shown in Fig. 3 (a), each supernode (frontal matrix) is assigned to a task and will be factorized in parallel. The green nodes are small nodes and the yellow represent big nodes.

For the assembly, factorization and pack of dense frontal matrices in each task, multifrontal sparse QR algorithm uses a modified version of strategy 3 from MA49 [2]. After the task assignment of the nodes in supernode tree (frontal matrix tree), a task tree, which is the data structure of tree-based parallelism, will be generated as shown on the (b) of Fig. 3. Based on this task tree, the main rule of dense QR factorization is that the condition of each task node execution must be satisfied— all of its child nodes have been accomplished. In Fig. 3 (b), root node 6 needs to wait for task 1 and task 4 to complete their jobs. Similarly, only after task 2 and task 3 are finished, task 4 can start.

2.4 Potential challenges

First of all, it's found from the matrix reordering operation that the performance of multifrontal QR factorization is heavily affected by the nonzero pattern. An excellent reordering algorithm can transform the sparse matrix into a satisfactory nonzero pattern to obtain a data structure that is more conducive to parallelism. A common choice used in SuiteSparseQR is to employ a default reordering

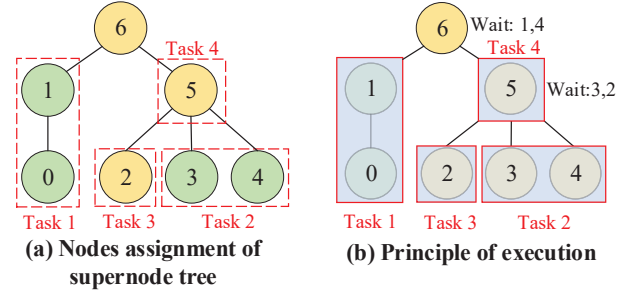


Figure 3: An example of nodes assignment and the principle of parallel executing in a frontal matrix tree.

algorithm based on experience or select a method that produces the lowest number of fill-in elements, which means all approaches need to be tried. However, the optimal ordering algorithm varies case by case and trying all algorithms consumes time and memory. Consequently, there is an urgent need to develop an efficient method to obtain the optimal nonzero pattern of a sparse matrix.

Secondly, based on the execution principle of tasks, SuiteSparse uses the batch task scheduling strategy of Intel Threading Building Blocks (TBB), which is not optimized for multicore architecture. This is mainly reflected in the fact that with the increase of sub-tasks and cores, the batch scheduling strategy cannot dynamically expand the parallelism of calculations, especially for recursive task structures. Therefore, we need to update the scheduling strategy to improve the parallelism of task nodes during the numerical phase.

Finally, the multi-NUMA cores devices are increasingly being adopted, but the existing multifrontal QR factorization lacks the optimization approaches on the NUMA architecture. In terms of the characteristics of NUMA architecture, the tree-level parallel framework needs to be further optimized to schedule threads and data, reasonably and efficiently. The above three challenges can be briefly summarized as:

- How to generate the optimal nonzero pattern of a sparse matrix for multifrontal QR factorization?
- How to improve the parallelism of the task nodes in the supernode tree?
- How to improve the computational efficiency of each task node for the NUMA architecture?

In the next three sections, we present our solutions to these questions respectively.

3 ADAPTIVE CLASSIFIER BY GCN

The traditional evaluation criteria of matrix nonzero pattern simply rely on the number of fill-in elements brought by the algorithm. We experimented with several sparse matrices randomly selected from the UF dataset and summarized in Table 1, where the minimum number of nonzero elements (nnz) is indicated in *italics* and the shortest factorization time is recorded in **bold**. From the results, the least amount of fill-ins does not guarantee the best performance, and when the number of fill-ins is the same (dataset *reorientation_8*), the performance is also very different. Since describing the independence and parallelism of a matrix nonzero patterns is an abstract

Table 1: Fill-ins and factorization time of the datasets randomly selected

Name	AMD		COLAMD		METIS		NESDIS	
	nnz	time	nnz	time	nnz	time	nnz	time
dwt_992	71560	0.006845	62392	0.009614	79048	0.009784	69596	0.010153
lms_3937	396808	0.031759	381582	0.040469	432566	0.064204	431138	0.066465
bcsstk14	280168	0.024935	386761	0.043955	288495	0.02656	331050	0.038921
epb1	1072482	0.055423	1190006	0.061124	1018745	0.160474	983713	0.169789
reorientation_8	4270506	0.256708	4270234	0.227302	4806331	0.275199	4270234	0.209945

and difficult problem, we decided to use machine learning methods as our solution. Specifically, our idea is to automatically select the optimal matrix reordering algorithm from AMD, COLAMD, METIS and NESDIS, via a well-trained classification model, which is equivalent to a matrix classification problem.

3.1 Graph convolution neural network

The challenge of this problem is how to make the classification model capture the nonzero pattern of a matrix (graph). When dealing with graph structured data, traditional machine learning applications usually need a preprocessing step to compress the graph structured information into a vector of reals [25], or encode the underlying data using topological relationships among the nodes [16]. However, the topological dependency of structured information may be lost during these preprocessing techniques so that the final result might be influenced. Therefore, in order to better obtain the matrix structured information and node dependency, we use the spatial-based graph convolutional neural network (GCN) as a machine learning classifier.

The idea of spatial-based graph convolutional neural network is mainly derived from the convolution of an image by traditional CNN [38]. The spatial-based method defines graph convolutions based on spatial relations of nodes, which is similar to the dependencies of rows or columns in eliminating process. A sparse matrix can be regarded as a special form of a graph, in which each nonzero element represents an edge, and each row or column represents a vertex. Therefore, for the input of the GCN model, the complete matrix data will split into two parts: edge feature set and vertex feature set. As shown in Fig. 4, the spatial GCN consists of a stack of neural network layers, each layer aggregates central node and its neighborhood information along edges, i.e., features of neighbors, and then passes this information to next layer, where we believe an appropriate model can also glean the relationship between parallelisms and dependencies of nodes and pass to neighborhood. The model can be divided into two phases: a message propagation phase and a readout phase. The message passing phase [35], namely spatial graph convolution, can be described as:

$$\begin{aligned}
 m_v^{(k)} &= \sum_{u \in \mathcal{N}(v)} M_k(f_v^{(k-1)}, f_u^{(k-1)}, e_{uv}) \\
 f_v^{(k)} &= U_k(f_v^{(k-1)}, m_v^{(k)}),
 \end{aligned} \tag{2}$$

where f_v is the hidden feature vector of node v ($f_v^{(0)} = x_v$), $M_k(\cdot)$ and $U_k(\cdot)$ are message function and vertex update function, $\mathcal{N}(v)$ is the set of neighbor nodes of node v , e_{uv} represents the features

of the edge from u to v . In readout phase, the representation of the entire graph is computed by readout function $R(\cdot)$ according to:

$$f_{\text{Graph}} = R(f_v^{(K)})|v \in \mathcal{G}, \tag{3}$$

where K denotes the last layer. It can solve various practical problems by assuming different forms of $U(\cdot)$, $M(\cdot)$, $R(\cdot)$ in GCN models.

3.2 Ground truth

The parallelism of the nonzero pattern are most directly reflected in the performance of the final numerical factorization process. Therefore, we decided to use the factorization time as an evaluation metric for the quality of a reordering algorithm. We performed multifrontal QR factorization on different reordering methods, and selected the optimal reordering method according to the time of numerical factorization. Because when dealing with a series of matrices with an identical nonzero pattern but diverse numerical values, symbolic analysis needs to be performed only once. This situation often occurs in practical problems, such as sub-matrices in the nested dissection on incorporating fast $\mathcal{H}$ -algebra[20], power grid's structure[7], etc. After choosing the optimal reordering algorithm for each matrix, we used a natural number to represent an algorithm, which serves as the ground truth.

3.3 Feature selection

Since the edges of the graph are sufficient to make good use of the positional relationship during information propagation, no extra features are assigned to the edges. However, vertices represent rows/columns, which are the key objects of all reordering algorithms, and the initial features of each vertex in graph $\mathcal{G}$ greatly affects training efficiency. Therefore, we assign three features (i.e. in-degree, out-degree, and *external degree*) to each vertex in order to analogize the minimum degree elimination algorithm[19]. All of these features are critical factors considered in the relevant heuristic reordering algorithms[1, 12]. The external degree of a vertex represents the number of edges produced after the node is eliminated in graph $\mathcal{G}$, which is also the number of fill-in in corresponding matrix $A^T A$. It can be computed by Algorithm 1.

As shown in Fig. 4, after extracting the information of vertices and edges through the GCN layers, we add some classic properties of the entire graph as extra information for the next multi-layer perceptron. These properties, as shown below, can capture the major characteristics of the matrix from the graph level, which are helpful for more accurate implementation of the classifier.

- n : the total number of nodes (matrix dimension);

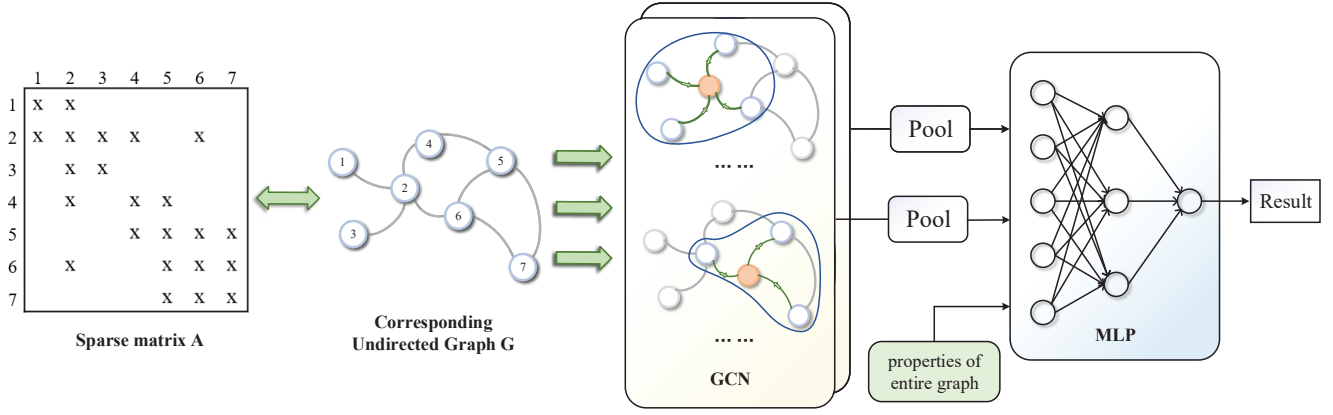


Figure 4: The structure of the classifier of auto-select reorder algorithm with graph convolutional network. Two layers of GCN play a role in information enhancement.

Algorithm 1 Calculation of *external degree*

Input: Vertex set $\mathcal{V}$, edge set $\mathcal{E}$, in-degree d_{in_i} and out-degree d_{out_i} of node v_i .

Output: The *external degree* $ed_1, ed_2, \dots, ed_n$ of each node.

```

1: for node  $v_i$  in  $\mathcal{V}$  do
2:    $ed_i = d_{in_i} \times d_{out_i}$ ;
3:   Store all nodes pointing to node  $v_i$  to array  $Neighbor_{in_i}$ ;
4:   Store all nodes pointed by node  $v_i$  to array  $Neighbor_{out_i}$ ;
5:   for every pair of  $(u, v)$  that  $u \in Neighbor_{in_i}$  and  $v \in Neighbor_{out_i}$  do
6:     if  $(u, v) \in \mathcal{E}$  then
7:        $ed_i = ed_i - 1$ ;
8:     end if
9:   end for
10: end for

```

- e : the total number of edges (number of nonzeros);
- $dense$: the sparsity of sparse matrix;
- avg_d : the average degree of a node;
- Fre_{in} : proportion of the largest in-degree nodes;
- Fre_{out} : proportion of the largest out-degree nodes;
- max_in : the maximum in-degree of a node;
- max_out : the maximum out-degree of a node.

The implementation details of our GCN classifier will be introduced in the subsequent experimental section.

4 IMPROVING THE PARALLELISM OF NUMERICAL FACTORIZATION

In the context of the second challenges listed in Section 2.4, SuiteSparseQR schedules and handles tasks tree-based parallelism by using Intel TBB. However, there are some drawbacks to this framework: (1) Tree-level parallelism and BLAS parallelism can conflict with each other and make a poor use of system resources [11]. (2) The parallelism of task scheduling strategy still has room for further

improvement in the current multicore architecture (especially on the NUMA architecture) and multi-task application environment.

4.1 Reducing interference between thread pools

On the first drawback mentioned above, the opportunity of BLAS parallelism arises in each frontal matrix. In fact, the time consumption is mainly concentrated on the QR factorization of the dense matrices that relies on Level-3 BLAS. But most of vendor-supplied BLAS, such as Goto BLAS, Automatically Tuned Linear Algebra Software (ATLAS), Intel Math Kernel Library (MKL) BLAS, as well as OpenBLAS has its own threads schedule block based on shared-memory multicore architecture, e.g. OpenMP and thread pool constructed with pthread library. None of the thread frameworks used by current popular BLAS libraries are TBB. The threads created by the upper TBB is separated from the threads created by the lower BLAS library, which results in a situation that the maximum number of threads during the numerical factorization is the product of these two sets of threads [11].

In order to solve this problem, we have completed a tasks scheduling framework based on the Pthread library, which retains the core requirements in the task scheduling process: synchronization, task queue, cyclic waiting, etc. The final results show that under our new scheduling framework and OpenBLAS library, the maximum number of threads needed for factorizing is only the sum of two thread sets, which alleviates the conflict between the two-level thread pools.

4.2 Task stream processing optimization

Intel TBB provides a container named *task_list*, which can be used as a task collection. Each parent task uses *allocate_child()* function to create a new child task. Before spawning any child tasks, the parent task node must call *set_ref_count* as a subtask counter. Then, the parent node derives child tasks by calling the *spawn_&wait_for_all* and waits for them to complete. The overall process is a typical batch-scheduling mode that is shown in Fig. 5. As can be seen, in

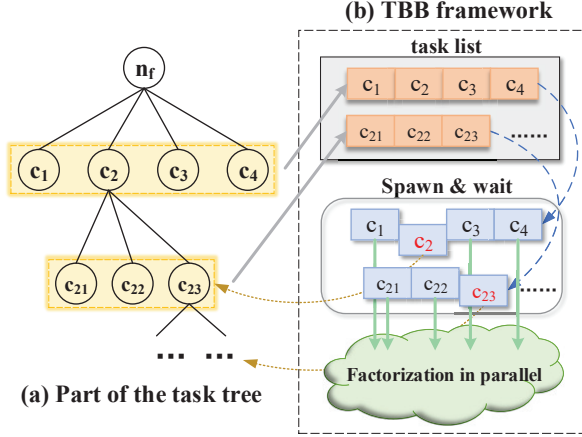


Figure 5: Batch scheduling strategy of Intel TBB. (a) The nodes in the light yellow box belong to a batch of jobs. (b) The batch scheduling framework of TBB.

the process of traversing the task tree, each root c_r of a subtree $T(c_r)$ will pack all the child nodes of its next layer into *task_list*. Then TBB will generate this batch of tasks at once, start parallel execution, and iterate this process until all nodes are completed.

The principle of our optimized parallel scheduling framework is shown in Fig. 6. In contrast, we cancel the rule of generating tasks in batches, and replace it with stream task processing, that is, every time a node c_r is traversed, the task of c_r is directly generated and the execution conditions are checked. If the conditions are satisfied, our framework starts to operate the task; otherwise, our method iterates this process until all child nodes of subtree $T(c_r)$ has been accomplished.

For the recursive task structures shown in Fig 3, this optimization strategy has at least the following two benefits: (1) Batch processing needs to wait for the arrival of a batch of tasks, while stream processing avoids this situation. (2) Compared with batch processing, the streaming strategy can make the independent subtasks (leaf nodes) start to calculate earlier. For example, when stream processing task scheduling is used, as for node c_2 , its child nodes c_{21} , c_{22} can be added to the thread pool earlier and computed in parallel with its siblings (c_3 and c_4). But in batch scheduling, a child node must wait for all the siblings of its parent node to join the thread pool. In most cases, streaming task scheduling strategy can improve the parallelism of tree-level tasks compared with Intel TBB.

5 TASK MAPPING OPTIMIZATION FOR NUMA ARCHITECTURE

Based on the structure of the task tree, each task is assigned to a thread for calculation during the numerical factorization. Therefore, how to perform efficient task mapping in a high-performance NUMA server is critical to improve the performance of each task. In this section, we implement two NUMA node affinity strategies: thread affinity and data affinity.

In NUMA architecture, cores in different nodes have different efficiencies in accessing memory. Generally, the time of remote

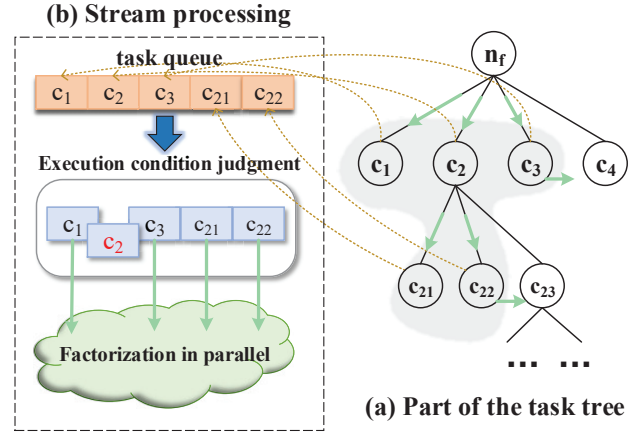


Figure 6: Stream scheduling strategy of our optimization. (a) The nodes covered gray shadow are in tree-level parallelism. (b) The optimized stream scheduling framework

access is much slower than that of local access, which is related to NUMA distance[34]. NUMA distance is only a relative distance between nodes that is determined by System Locality Information Table (SILT) inside the ACPI specification; where the values in this table range from 10 to 254 (255 is used for "no connection between these NUMA domains"). The value 10 represents how quickly a thread can access memory in the same NUMA node (the fastest case), and value 254 would be 25.4 times slower. For instance, the distance table of a 4 NUMA nodes architecture can be represented as follows:

$$\begin{matrix} & \text{node} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & & \begin{pmatrix} 10 & d_1 & d_2 & d_4 \\ d_1 & 10 & d_3 & d_2 \\ d_2 & d_3 & 10 & d_1 \\ d_4 & d_2 & d_1 & 10 \end{pmatrix} & = D. \end{matrix} \quad (4)$$

Based on the NUMA distance table, we propose our **first optimization** for thread affinity. We divide the threads into several groups, whose number is the same as the number of NUMA nodes, according to the characteristics of the NUMA module. The threads in each group are pinned to the entire set of cores in the NUMA node, thereby avoiding the increased context switching caused by the migration of threads on different NUMA modules. In particular, for this kind of recursive tree task structure, the subtasks own inherent affinity of their parents, which is conducive to the subsequent update of each front matrix.

What's more, when all threads in a group are busy and a new task arrives, the operating system will randomly select an idle thread in the thread queue to execute the task. In response to this issue, we design and implement an autonomous search for idle resources according to the NUMA distance from near to far in the scheduling module. The search sequence table for each NUMA node will be automatically generated according to SILT. For instance, suppose $d_1 < d_2 < d_3 < d_4$, then the search array corresponding to NUMA node 0 is $\{0, 1, 2, 3\}$, $\{1, 0, 3, 2\}$ for node 1, $\{2, 3, 0, 1\}$ for node 2 and $\{3, 2, 1, 0\}$ for node 3. In this way, the task-level thread framework

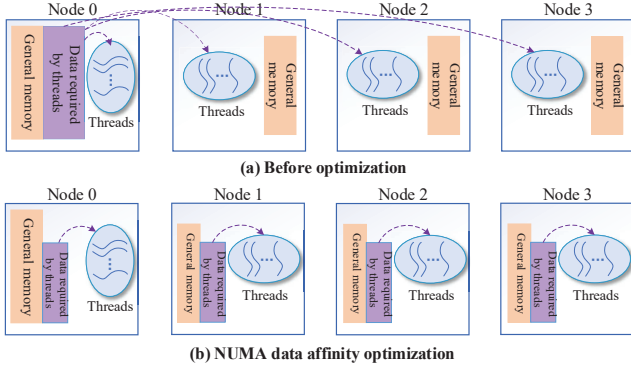


Figure 7: (a) Remote NUMA memory access mode and (b) NUMA data affinity optimization.

greatly reduces the number of remote accesses to the processor core when task scheduling is performed.

The second optimization opportunity occurs when threads are fetching data. Assuming the load of all cores is fully balanced, the data required by each thread needs to be copied from the same NUMA node (where the *main* function is located). Only considering the time of accessing data, the total time cost of all threads is:

$$\min_{j \in [0, N)} \frac{\sum_{i=0}^{N-1} D(i, j)}{10 \times N} T, \quad (5)$$

where N is the number of NUMA nodes, T represents the time required to access all data locally, $D(i, j)$ means the entry $d_{i,j}$ in distance matrix D . The whole process is shown in Fig. 7 (a). In Fig. 7, *General memory* represents the memory space used in the entire factorizing process in addition to the dense numerical factorization, which usually includes some index information, symbolic analysis information and thread identification number indexes, etc. The purple memory area is the data that the threads will require to operate when performing the decomposition task. As shown in Fig. 7, except for the threads in Node 0, other threads need to cross the NUMA node to fetch their data.

Therefore, our second NUMA affinity idea is: when the symbolic analysis allocates space for subsequent computing tasks, it will allocate space to different NUMA nodes according to the task number in advance, as shown in Fig. 7 (b). In this case, the threads in each NUMA node can access memory locally, which can greatly shorten the time for data fetching. The theoretical speedup ratio is:

$$\min_{j \in [0, N)} \frac{\{\max_{i \in [0, N)} D(i, j)\}}{10 \times N}. \quad (6)$$

Note that the *max* is used because the upper bound of overhead for accessing memory in parallel is only related to the maximum NUMA distance. In addition, the NUMA node that executes the *main* function will slightly affect the performance of the original remote NUMA access.

6 EXPERIMENTAL RESULTS

6.1 Experiments of the GCN classifier

6.1.1 Implementation details.

All experiments in this subsection are carried out on *Ubuntu 16.04.6 LTS* with TITAN RTX-24G. The GCN is implemented in python 3.7, PyTorch 1.4.0 and PyTorch-geometric 1.6.0. After conducting a large number of experimental tests and comprehensively considering the training cost and accuracy indicators, we decide to adopt a two-layer parallel GCN combined with a three-layer fully connected network. The basic structure of the classifier is shown in Fig. 4. The specific details of implementation are as follows:

- First, two layers of parallel GCN are implemented to obtain the hidden structural attributes of each node in the graph. The dimension of the first layer is (3, 128) and that of the second layer is (128, 128).
- Second, we record the structure features extracted from these two layers as x_1 and x_2 respectively. Additionally, we concatenate the extra information x_e of the entire graph with the structural features (x_1, x_2) and get $x = (x_1, x_2, x_e)$.
- Then, we take x as input to a multilayer perceptron with a structure of $(256 + size_{x_e}, 64, 32)$. The three fully connected layers can summarize the hidden information obtained before.
- Finally, we use the *softmax* function to complete the classification of the graph (sparse matrix).

After our test, the best algorithm of graph convolution for this problem is *Higher-order Graph Neural Networks* given in [30], whose new feature $f_v^{(k)}$ in k -th layer is computed as:

$$f_{v,s}^{(k)} = \sigma(f_{v,s}^{(k-1)} \cdot \Theta_1^{(k)} + \sum_{u \in N(v)} f_{u,s}^{(k-1)} \cdot \Theta_2^{(k)}), \quad (7)$$

where Θ_1, Θ_2 are learnable model parameter matrices, s is the size of the subset of neighbors to be aggregated and $\sigma(\cdot)$ denotes an activation function where we use **ReLU**.

The stochastic gradient descent method is used to train the weights matrices Θ_1 and Θ_2 . The loss function we used is negative log likelihood loss (nll-loss). Based on experimental results, we choose the following hyperparameter settings: *batch_size* = 4, *learning_rate* = 0.0001, *epochs* = 5000. The drop-out ratio in GCN layers is set to 0.2. The ratio of training set, validation set and test set is $\frac{2}{3} : \frac{1}{6} : \frac{1}{6}$. The training and testing experiments are repeated five times, and the best inference result is recorded each time.

6.1.2 Results of adaptive classifier.

We randomly selected 408 square matrices as our datasets, whose largest size is 10605 and the maximum number of nonzero entries is 424587, from *University of Florida Sparse Matrix Collection* (UF dataset). A program is written based on SuiteSparseQR to obtain the ground truth with four existing reordering approaches: AMD (8.09%), COLAMD (8.82%), METIS (19.12%), and NESDIS (63.97%). The accuracy of model during training and testing is shown in blue and red respectively in Fig 8, where the *nll-loss* is shown in brown. As we can seen from Fig 8, when the epochs reach 4500, the loss gradually converges to 0.05, and the accuracy of the model fluctuates around 80%.

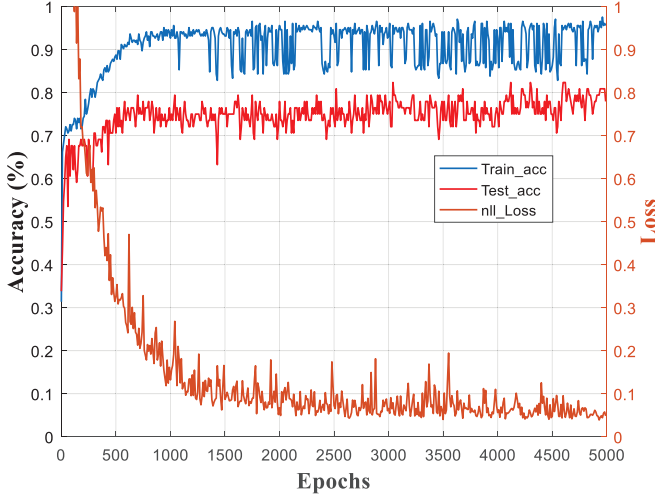


Figure 8: The results of GCN classifier.

The statistical results of our GCN are summarized in Table 2. In Table 2, the MLP-NN is a prior work proposed by Cui [9] to inference the best memory-saving reordering method in power grid analysis problem. Here *Accuracy* is the number of correct predictions divided by the total number of elements in the test set. *Average Mem.* refers to the average memory required by each sparse matrix in the data set during factorizing phase. *Average Time in Numeric* means the average time for this testing subset in numerical factorization after using the corresponding reordering algorithm. For comparison, the average memory costs and factorization time corresponding to the situation, where these four reordering approaches are always applied, are also provided. The *Time required* is the actual computing time for the entire testing subset in each classifier. The first main row of the last column gives the result of brute-force approach (running all 4 ordering algorithms).

Table 2: Results of different approaches on 408 matrices

	Accuracy (%)	Average Mem. (KB)	Average Time in Numeric(s)	Time required(s)
AMD	-	513.98	1.4057	0.9652
COLAMD	-	488.08	1.1560	
METIS	-	498.48	1.3115	
NESDIS	-	487.82	1.2368	
MLP-NN	60.32 ± 2.38	472.20	1.2576	0.3487
GCN	79.41 ± 2.94	488.91	0.9157	0.4408

As shown in Table. 2, our GCN classifier has an accuracy of nearly 20% higher than that of MLP-NN. As for the average memory required (the number of fill-in elements after reordering), our GCN classifier is slightly higher than MLP by 3.5%. During the numerical phase, after our classifier selects the optimal reordering algorithm, the average calculation time is reduced by 20.78% compared to the COLAMD called by SuiteSparseQR by default, and 27.19% compared with the MLP memory-saving approach.

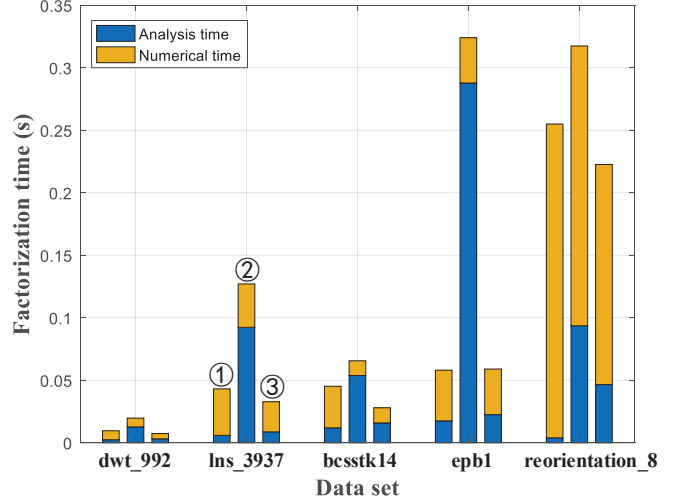


Figure 9: The total factorization time comparison of multifrontal QR on 5 sparse matrices by ① SuiteSparseQR default method, ② brute-force approach and ③ GCN classifier.

We used 5 sparse matrices listed in Table 1, and conducted experiments to test the application efficiency of GCN in actual sparse QR factorization. The results are shown in Fig. 9, in which the blue bar represents the symbolic analysis time, and the yellow represents the numerical factorization time. We compared the two traditional selection methods in SuiteSparse: ① SuiteSparseQR default method COLAMD, ② brute-force approach.

From Fig. 9, we can see that the brute-force method has the worst performance because it needs to run four reordering algorithms and solely depends on the number of fill-ins. It usually takes 0.3s to load a well-trained classifier, and the actual prediction cost is about 0.002s ~ 0.01s. In conclusion, compared with the prior method, the performance of our GCN classifier in the subsequent numerical factorization is improved by about 20.78%, and both memory and time overhead are better than brute-force method.

6.2 Optimization results in numerical phase

In this section, the efficiency of our optimization strategies implemented in numerical factorization will be demonstrated step by step. When testing with different sparse matrix data sets, we always compare their performance under the identical reordering algorithm (default COLAMD). Since the Intel MKL has remained the gold standard for high-performance numerical computing [3, 10, 33], we use the performance of its sparse QR function as a reference to compare our optimal STM-Multifrontal QR factorization. The experimental platforms are shown in Table 4.

All experiments related to multifrontal QR factorization are carried out on the Huawei TaiShan Server. The *Intel MKL 2020.0.166* on Intel Xeon Gold 6248 serves as the top performance benchmark for the current sparse QR decomposition. The 1864 square matrices from the UF dataset are used to verify the final performance and error. We randomly selected six sparse matrices in entire data sets, which are shown in Table 3, as a reference for specific performance

Table 3: Six Sparse matrix data sets used as examples in numerical factorization

#	name	$m/10^3$	$nnz/10^3$	Density	Pattern Symmetry	Description
1	Mycielski/mycielskian13	6.1	1227.7	3.253%	100%	Undirected Mycielskian graph M13
2	Tromble/language	399.1	1216.3	0.001%	5.80%	Finite-state machine for natural language processing
3	Schenk_IBMNA/c-67	57.9	531.9	0.016%	100%	Nonlinear Optimization Problem Sequence
4	FEMLAB/sme3Dc	42.9	3148.7	0.171%	100%	3D structural mechanics problem
5	Ronis/xenon1	48.6	1181.1	0.050%	100%	Materials Problem
6	Martin/marine1	400.0	6226.5	0.004%	92.20%	Chemical Oceanography Problem

¹ where m is size of matrix, nnz is the number of nonzeros.

² Pattern Symmetry only describes the degree of symmetry of the position of nonzeros, regardless of the values.

Table 4: Platforms for SuiteSparseQR optimization and MKL

	Huawei TaiShan Server	Intel Xeon Gold 6248
Processor	ARM v8.2 architecture, 2 Kunpeng 920	Intel(R) Xeon(R) Gold 6248 CPU
Clock cycle	2.6 GHz	3.9 GHz
Memory	256 GB	128 GB
NUMA node(s)	4 NUMA, 128 cores	1 NUMA, 20 cores
Operating system	CentOS Linux 4.18.0.aarch64	Linux ubuntu 4.4.0-142-generic
Vector register	128bit×2	512bit×2
Theoretical throughput	1331.2 Gflops	1248 Gflops
HPL benchmark	2.2407 Gflops	13.87 Gflops

analysis in the optimization process. The error verifications in this section are based on the following steps:

- (1) First, according to the equation system $Ax = b$, we first assign x_{ori} as $x_{ori} = [0, 1, 2, \dots, m]$.
- (2) The second step is to calculate the vector b on the right side of the equation system by the equation $b = A \cdot x_o$.
- (3) Then, we obtain Q and R after the QR factorization, and next we obtain $x_{real} = R^{-1}Q'b$. Finally, we can calculate the error by comparing the difference between x_{ori} and x_{real} by:

$$e = \frac{\|x_{real} - x_{ori}\|_2}{m}. \quad (8)$$

6.2.1 Optimization results of streaming task scheduling.

When performing sparse QR factorization on the TaiShan server, we observed through testing that the system will deliver the optimal performance when the number of OpenBLAS threads is set to 32, which is also the number of cores in each NUMA node. Next, we respectively tested the performance of SuiteSparseQR-5.7.2 without tree-level parallelism, with the TBB for task tree parallelism, and with our optimized streaming parallel strategy for parallelism. The results are summarized in Table 5.

Table 5: Numerical factorizing performance of 6 datasets

#	ntasks	without tree-level parallelism		TBB: 128		Streaming task scheduling:128	
		time	Gflops	speed up	Gflops	speed up	Gflops
1	1	3.37	76.6	1.04	76.7	1.10	77.2
2	8	1861.60	222.8	1.05	232.9	1.03	232.8
3	55	4.63	54.9	1.24	58.0	1.39	64.0
4	12	5.17	32.1	1.61	47.0	1.72	48.1
5	82	6.25	49.5	1.80	85.6	1.85	87.4
6	212	128.85	75.72	1.43	104.0	1.47	106.8

Table 5 shows the results of those six sparse matrices on the TaiShan Server. Note that the execution time (only including numerical factorization time) is in seconds in the case of without tree-level parallelism (the single-thread mode). Gflops is calculated by dividing the total number of floating-point operations with the factorization time (including symbolic and numerical factorization time). *TBB:128* represents that the Intel TBB automatically detects 128 cores of this machine and creates 128 threads for tree-level parallelism. The third main column also shows the performance of numerical factorization under our optimized streaming scheduling framework (128 threads, too) mentioned in Section 4. The *speedup* is the ratio of time relative to single-threaded mode. The *ntasks* is the number of nodes in the task tree.

On 1864 datasets, our streaming optimization methods for parallelism achieve higher performance than TBB by outperforming the speedup up to 2.03x (with an average of 1.05x). We observed that when the structure of the task tree is dumpy rather than lanky, stream processing can effectively avoid idle waiting time between sibling nodes caused by batch strategy.

6.2.2 Results of task mapping optimization.

Next, we introduce the NUMA affinity mechanism to our streaming processing thread module to achieve better task mapping. In numerical factorization, data migration mainly occurs in the assembling and packaging of the elements in each stack of the thread. The NUMA distance table of TaiShan Server is:

$$node \quad \begin{matrix} 0 & 1 & 2 & 3 \\ 0 & \begin{pmatrix} 10 & 16 & 32 & 33 \\ 16 & 10 & 25 & 32 \\ 32 & 25 & 10 & 16 \\ 33 & 32 & 16 & 10 \end{pmatrix} \end{matrix} \quad (9)$$

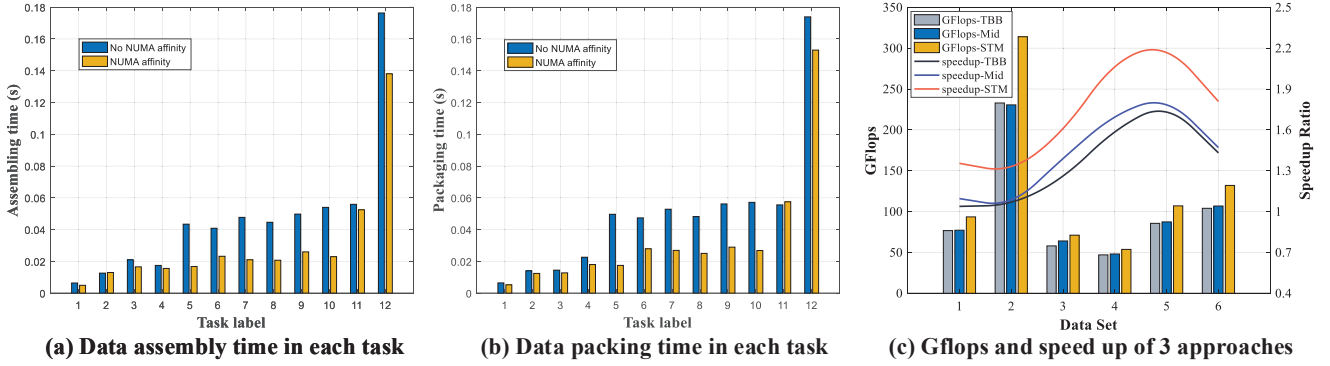


Figure 10: (a) The data assembly time and (b) data packaging time of the stacks in 12 tasks of *Sme3Dc.mtx*. Subgraph (c) is a comparison of the specific performance of the five data sets in Table 3 under each parallel method. The bar graph represents *Gflops*, and the curve is the *speedup*. The reordering method uses the default COLAMD.

According to (6), the theoretical data access speed is up to 3.2 times faster. We took the *sme3Dc* dataset including 12 subtasks as an example to test the optimization effect of task mapping, the results are shown in Fig. 10 (a) and (b). As shown in Fig. 10, after implementing the task mapping optimization strategy, the speed of data assembly and data packaging both have been improved, and the range of speedup ratio is 1.1x ~ 2.8x which is consistent with theoretical analysis.

The results of numerical factorization time of these 6 sample data sets are listed in Table 6. We have added an additional set of comparative experiments that pin each thread to a single core, which is a traditional NUMA core affinity scheme. We can see that the task mapping optimization achieves an average speedup of 1.73x over single-thread mode. The *Gflops* of the multifrontal QR factorization is also improved by 1.10x ~ 1.35x. In contrast to the traditional core affinity mode, the NUMA node affinity strategy has an average speedup of 1.08x across these 6 data sets. This is because it allows for some leeway if the OS considers it better to move a thread to a different core of the same node[32].

Table 6: Cumulative results of implementing NUMA affinity

#	without NUMA affinity		Core affinity		NUMA node affinity	
	speed up	Gflops	speed up	Gflops	speed up	Gflops
1	1.10	77.2	1.28	94.0	1.35	99.2
2	1.03	232.8	1.26	275.3	1.29	285.9
3	1.39	64.0	1.47	66.9	1.58	71.1
4	1.72	48.1	1.86	47.5	2.11	52.6
5	1.85	87.4	1.88	87.7	2.24	103.1
6	1.47	106.8	1.74	125.4	1.81	130.2

Fig. 10 (c) shows the two-step optimization effect compared to the performance of SuiteSparseQR with TBB parallel framework, where *Mid* represents the results that only stream scheduling optimization is implemented and *STM* represents the results of implementing both stream scheduling and task mapping optimization strategies. In tree-level parallelism, the performance of the *Mid* version is only

slightly improved compared with the original SuiteSparseQR. However, according to the characteristics of NUMA architecture, after implementing task mapping optimization, the overall numerical factorization speed is 20% ~ 30% faster than SuiteSparseQR under TBB. The *Gflops* of our STM-Multifrontal QR factorization process also has an improvement of 1.15x to 1.35x (average 1.27x).

6.2.3 Compare STM-MQR and MKL sparse QR.

Next, we compare our STM-Multifrontal QR with the MKL sparse QR function on 1864 data sets. Among those, we reported the results of 1691 data sets in which both function interfaces can run successfully. From Table 4, it can be seen that the theoretical calculation peaks of these two machines are close but the actual floating-point computing power (tested by HPL benchmark) of these two servers is six times different.

Fig. 11 (a), (b), and (c) are performance comparisons by matrix size, in which the blue dot above the red dotted line indicates one data set with better performance than MKL and vice versa. When the matrix size is less than 10^4 , the performance of STM-Multifrontal QR factorization can surpass the performance of MKL, while as the matrix size increases, the performance difference between the two functions on some data sets starts to manifest.

Fig. 11 (d), (e), and (f) show error comparisons between two functions, where each orange dot represents one error result of STM-Multifrontal QR and the blue dots represent that of MKL QR function. It can be seen that the overall position of the orange points on the y-axis is higher than the blue points, which indicates that the accuracy of our STM-Multifrontal QR algorithm is higher than that of MKL in most cases of the dataset. The statistical results of performance and error will be shown in the next section.

6.3 Results of STM-MQR empowered by GCN

6.3.1 Performance and error.

In this section, three techniques are turned on to test the performance of STM-Multifrontal QR empowered by GCN. The complete factorization time of the single-thread version of *SuiteSparseQR-2.0.9* is used as the baseline. Additionally, we tested the Sparse QR function of *Eigen-3.3.7*, a commonly used open-source C++ library

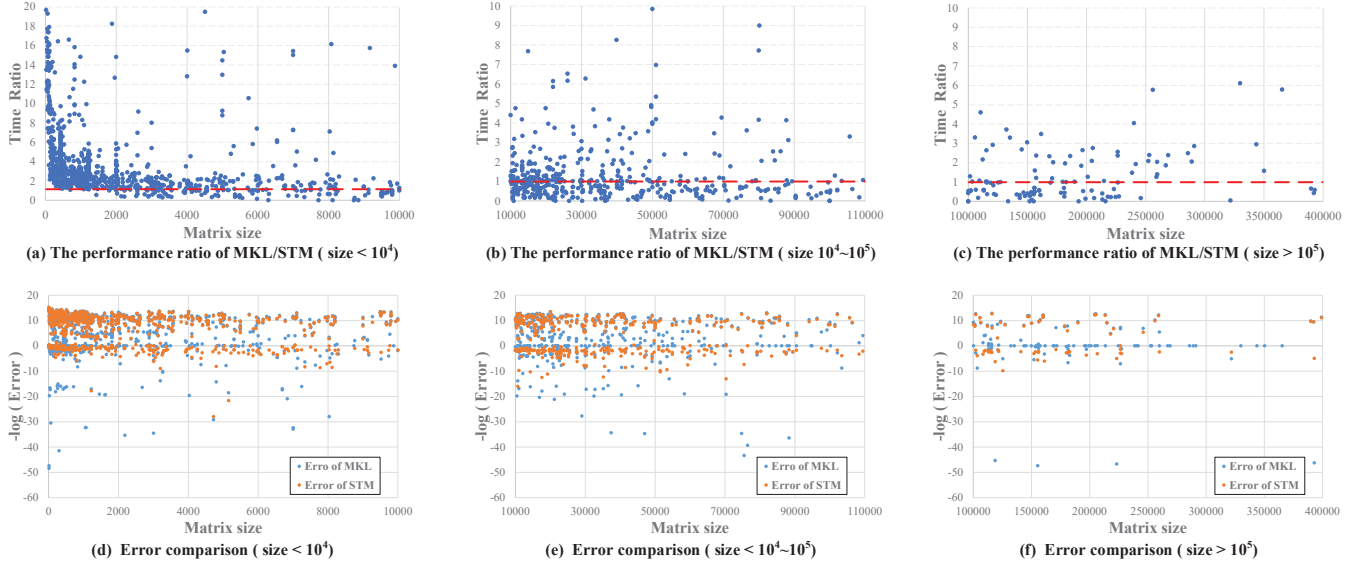


Figure 11: (a),(b) and (c) present scatter plots of Sparse QR factorization time ratio (MKL/STM), where $y > 1$ means that the STM-Multifrontal QR performance is better. (d),(e) and (f) show error verification ($-\log_{10}(Error)$) comparison between two sparse QR factorization algorithms (The data set is sorted by size from small to large).

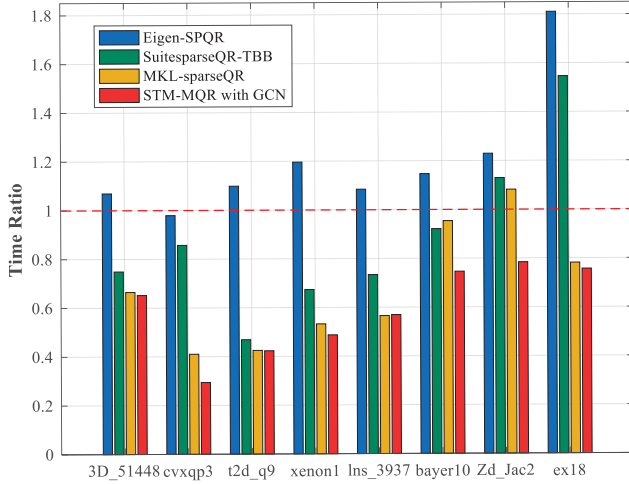


Figure 12: The comparison results of STM-MQR empowered by GCN. $Time\ Ratio = fac_time/baseline_time$.

[24]. The current optimal version of SuitesparseQR (with TBB parallelism), and the MKL sparse QR tested before are also used for comparison. Eight data sets are randomly selected, and the comparison results between these libraries are shown in Fig. 12, where $Time\ ratio$ is the ratio of their factorization time to the baseline.

As shown in Fig. 12, STM-MQR empowered by GCN has a higher speedup than other libraries. It can be seen that when the TBB yields negative impact (such as data set *Zd_Jac2* and *ex18*), our optimization technique still achieves a significant acceleration effect.

Table 7: Statistical results of STM-MQR empowered by GCN

Compared with SuiteSparseQR		
Number of comparable matrix	1864	The proportion
Speedup ratio ≥ 1	1442	77.36%
Speedup ratio $\leq 90\%$	196	10.51%
Error (SuitesparseQR/STM-MQR)		
≤ 0.1	17	0.9%
≥ 1	1478	79.29%
Compared with MKL		
Number of comparable matrix	1691	The proportion
Speedup ratio ≥ 1	1277	75.52%
Speedup ratio $\leq 90\%$	381	22.53%
Error range	MKL	STM-MQR
$\leq 10^{-4}$	982	88.70%
$\leq 10^{-6}$	823	96.47%

Table 7 shows the statistical results compared with the current optimal performance SuitesparseQR (TBB turned on).

It can be seen that 77.36% of the datasets have been accelerated compared to the original algorithm, while only nearly 10% of the datasets exhibit slowdown. For these decelerated datasets, the reason we speculate is that the acceleration of the numerical factorization fails to offset the additional GCN calculation overhead. In terms of error, only 1% of the datasets expand the error by an order of magnitude. Overall, our three novel optimization techniques achieve satisfactory results. Without compromising accuracy, the average speedup on 1864 datasets is 1.22x (up to 6.38x).

In addition, Table 7 also summarizes the comparison results with MKL, where we can see that the performance of STM-MQR

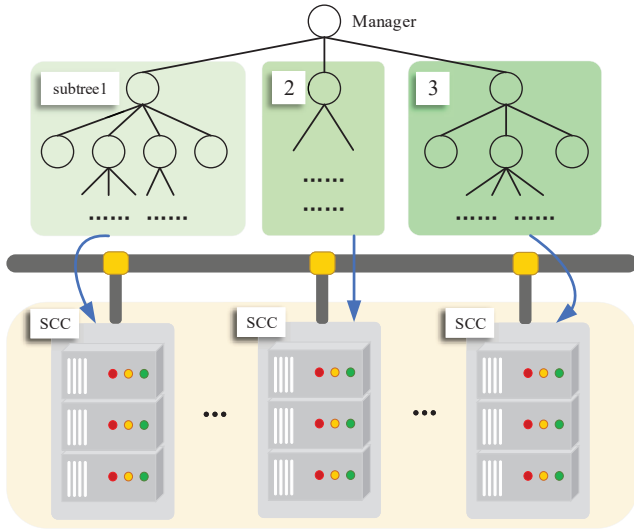


Figure 13: The scalability of multifrontal QR factorization in distributed-memory architecture.

can surpass MKL on nearly 80% of datasets. Especially for high-precision data sets (error $\leq 10^{-6}$), this ratio is further increased. From the *Error range*, we can see that the accuracy of STM-MQR is similar to the MKL standard, and nearly 95% of the data sets are at the same level. In summary, STM-MQR empowered by GCN is an potential substitute for MKL sparse QR in specific problems.

6.3.2 Discussion of scalability.

With the expansion of matrix size, extending existing factorization algorithms to distributed memory architecture has attracted more and more attention[14]. The multifrontal QR algorithm has natural scalability for distributed architecture because of its own supernode tree structure. As can be seen from Fig. 13, a large-scale task tree can be divided into multiple subtree jobs, which are submitted to each Super Computing Cluster (SCC) for calculation. Then the tasks in the same subtree are allocated within the nodes of a cluster. This pattern of task assignment limits the communication range of subtasks to their respective clusters. For each SCC, the stream processing strategy can avoid the invalid waiting time between nodes caused by batching processing, which is more prominent in network communication. In symbolic analysis, the manager node can pre-configure each subtree data in the distributed memory, thereby reducing the data fetching overhead of each node. Therefore, further research is needed especially on tree-level task division, node communication, and data management for distributed-memory STM-MQR.

7 CONCLUSION AND FUTURE WORK

This paper improves the SuiteSparseQR multifrontal factorization algorithm in three aspects. In symbolic analysis, it proposes the use of GCN to infer the optimal reordering algorithm from several candidates. The experiments show that the accuracy of GCN classifier is as high as 79.41%, and it can improve the performance of the numerical stage by 20.78% compared to the default method.

In the numerical factorization stage, it provides an optimization scheme for streaming task mapping for NUMA architecture, which we call STM-Multifrontal QR factorization. The results reveal that, compared with the current optimal performance of SuiteSparseQR, STM-Multifrontal QR empowered by GCN achieves an average speedup of 1.22x without compromising accuracy. Nearly 80% of matrices from the UF dataset can reach the same or even better level line compared with MKL sparse QR.

In future work, we plan to explore more comprehensive design space of GCN (e.g. topology, feature selection) for better accuracy and efficiency. In addition, we plan to apply our proposed techniques to other methods that process sparse matrices. Furthermore, we will investigate mechanisms, such as unifying the threads framework of tree-level parallelism and BLAS, and extending STM-Multifrontal QR to distributed memory architecture, which have great potential for exascale matrix processing.

ACKNOWLEDGMENTS

We thank all reviewers for their valuable feedbacks that help on improving the quality of our work. We thank Dr. Tao Li for his significant contribution to this work. The research was partially funded by the National Key R&D Program of China (Grant Nos. 2018YFB0204302), the Key Program of National Natural Science Foundation of China (Grant No. 92055213), and the National Natural Science Foundation of China (Grant Nos. 61872127 and 61751204).

REFERENCES

- [1] Patrick R Amestoy, Timothy A Davis, and Iain S Duff. 1996. An approximate minimum degree ordering algorithm. *SIAM J. Matrix Anal. Appl.* 17, 4 (1996), 886–905.
- [2] Patrick R Amestoy, Iain S Duff, and Chiara Puglisi. 1996. Multifrontal QR factorization in a multiprocessor environment. *Numerical linear algebra with applications* 3, 4 (1996), 275–300.
- [3] Michael Anderson, Grey Ballard, James Demmel, and Kurt Keutzer. 2011. Communication-avoiding QR decomposition for GPUs. In *2011 IEEE International Parallel & Distributed Processing Symposium*. IEEE, 48–58.
- [4] Cleve Ashcraft and Roger Grimes. 1989. The influence of relaxed supernode partitions on the multifrontal method. *ACM Transactions on Mathematical Software (TOMS)* 15, 4 (1989), 291–309.
- [5] Alfredo Buttari. 2013. Fine-grained multithreading for the multifrontal QR factorization of sparse matrices. *SIAM Journal on Scientific Computing* 35, 4 (2013), C323–C345.
- [6] Léopold Cambier, Chao Chen, Erik G Boman, Sivasankaran Rajamanickam, Raymond S Tuminaro, and Eric Darve. 2020. An algebraic sparsified nested dissection algorithm using low-rank approximations. *SIAM J. Matrix Anal. Appl.* 41, 2 (2020), 715–746.
- [7] Stephen Cauley, Venkataramanan Balakrishnan, and Cheng-Kok Koh. 2010. A parallel direct solver for the simulation of large-scale power/ground networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 29, 4 (2010), 636–641.
- [8] Thomas F Coleman, Anders Edenbrandt, and John R Gilbert. 1986. Predicting fill for sparse orthogonal factorization. *Journal of the ACM (JACM)* 33, 3 (1986), 517–532.
- [9] Ganqu Cui, Wenjian Yu, Xin Li, Zhiyu Zeng, and Ben Gu. 2019. Machine-learning-driven matrix ordering for power grid analysis. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 984–987.
- [10] Scott Cyphers, Arjun K Bansal, Anahita Bhiwandiwala, Jayaram Bobba, Matthew Brookhart, Avijit Chakraborty, Will Constable, Christian Convey, Leona Cook, Omar Kanawi, et al. 2018. Intel ngraph: An intermediate representation, compiler, and executor for deep learning. *arXiv preprint arXiv:1801.08058* (2018).
- [11] Timothy A Davis. 2011. Algorithm 915, SuiteSparseQR: Multifrontal multi-threaded rank-revealing sparse QR factorization. *ACM Transactions on Mathematical Software (TOMS)* 38, 1 (2011), 1–22.
- [12] Timothy A Davis, John R Gilbert, Stefan I Larimore, and Esmond G Ng. 2004. Algorithm 836: COLAMD, a column approximate minimum degree ordering algorithm. *ACM Transactions on Mathematical Software (TOMS)* 30, 3 (2004), 377–380.

- [13] Ove Edlund. 2002. A software package for sparse orthogonal factorization and updating. *ACM Transactions on Mathematical Software (TOMS)* 28, 4 (2002), 448–482.
- [14] Srinivas Eswar, Koby Hayashi, Grey Ballard, Ramakrishnan Kannan, Richard Vuduc, and Haesun Park. 2020. Distributed-memory parallel symmetric non-negative matrix factorization. In *2020 SC20: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE Computer Society, 1041–1054.
- [15] Leslie V Foster and Timothy A Davis. 2013. Algorithm 933: Reliable calculation of numerical rank, null space bases, pseudoinverse solutions, and basic solutions using SuiteSparseQR. *ACM Transactions on Mathematical Software (TOMS)* 40, 1 (2013), 1–23.
- [16] Paolo Frasconi, Marco Gori, and Alessandro Sperduti. 1998. A general framework for adaptive processing of data structures. *IEEE transactions on Neural Networks* 9, 5 (1998), 768–786.
- [17] Alan George and Michael T. Heath. 1980. Solution of sparse linear least squares problems using givens rotations. *Linear Algebra Appl.* 34 (1980), 69–83. [https://doi.org/10.1016/0024-3795\(80\)90159-7](https://doi.org/10.1016/0024-3795(80)90159-7)
- [18] Alan George, Joseph Liu, and Esmond Ng. 1988. A data structure for sparse QR and LU factorizations. *SIAM J. Sci. Statist. Comput.* 9, 1 (1988), 100–121.
- [19] Alan George and Joseph WH Liu. 1989. The evolution of the minimum degree ordering algorithm. *Siam review* 31, 1 (1989), 1–19.
- [20] Pieter Ghysels, Xiaoye S Li, François-Henry Rouet, Samuel Williams, and Artem Napov. 2016. An efficient multicore implementation of a novel HSS-structured multifrontal solver using randomized sampling. *SIAM Journal on Scientific Computing* 38, 5 (2016), S358–S384.
- [21] John R Gilbert, Xiaoye S Li, Esmond G Ng, and Barry W Peyton. 2001. Computing row and column counts for sparse QR and LU factorization. *BIT Numerical Mathematics* 41, 4 (2001), 693–710.
- [22] Abeynaya Gnanasekaran and Eric Darve. 2020. Hierarchical Orthogonal Factorization: Sparse Square matrices. *arXiv preprint arXiv:2010.06807* (2020).
- [23] Gene H Golub and Charles F Van Loan. 2013. *Matrix computations*. Vol. 3. JHU press.
- [24] Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen v3. <http://eigen.tuxfamily.org>.
- [25] Simon Haykin and N Network. 2004. A comprehensive foundation. *Neural networks* 2, 2004 (2004), 41.
- [26] MT Heath and Danny C Sorensen. 1986. A pipelined Givens method for computing the QR factorization of a sparse matrix. *Linear algebra and its applications* 77 (1986), 189–203.
- [27] Joseph WH Liu. 1986. On general row merging schemes for sparse Givens transformations. *SIAM journal on scientific and statistical computing* 7, 4 (1986), 1190–1211.
- [28] Joseph WH Liu. 1992. The multifrontal method for sparse matrix solution: Theory and practice. *SIAM review* 34, 1 (1992), 82–109.
- [29] Pontus Matstoms. 1995. Parallel sparse QR factorization on shared memory architectures. *Parallel Comput.* 21, 3 (1995), 473–486.
- [30] Christopher Morris, Martin Ritzert, Matthias Fey, William L Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2019. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4602–4609.
- [31] Daniel J Pierce and John G Lewis. 1997. Sparse multifrontal rank revealing QR factorization. *SIAM J. Matrix Anal. Appl.* 18, 1 (1997), 159–180.
- [32] Michael Voss, Rafael Asenjo, and James Reinders. 2019. TBB on NUMA Architectures. In *Pro TBB*. Springer, 581–604.
- [33] Endong Wang, Qing Zhang, Bo Shen, Guangyong Zhang, Xiaowei Lu, Qing Wu, and Yajuan Wang. 2014. Intel math kernel library. In *High-Performance Computing on the Intel® Xeon Phi™*. Springer, 167–188.
- [34] Sean Williams, Latchesar Ionkov, and Michael Lang. 2017. Numa distance for heterogeneous memory. In *Proceedings of the Workshop on Memory Centric Programming for HPC*. 30–34.
- [35] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* (2020).
- [36] Mihalís Yannakakis. 1981. Computing the minimum fill-in is NP-complete. *SIAM Journal on Algebraic Discrete Methods* 2, 1 (1981), 77–79.
- [37] Sencer Nuri Yeralan, Timothy A Davis, Wissam M Sid-Lakhdar, and Sanjay Ranka. 2017. Algorithm 980: Sparse QR factorization on the GPU. *ACM Transactions on Mathematical Software (TOMS)* 44, 2 (2017), 1–29.
- [38] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2018. Graph neural networks: A review of methods and applications. *arXiv preprint arXiv:1812.08434* (2018).

Appendix: Artifact Description/Artifact Evaluation

SUMMARY OF THE EXPERIMENTS REPORTED

In Section 6.1, we ran the training and validation process of the GCN classifier on TITAN RTX-24G using the PyTorch-geometric package(v1.6.0) as described in the paper. In Section 6.2, all experiments related to multifrontal QR factorization were run on Taishan 2280 V2 Server (ARMv8 architecture, 4 NUMA nodes) using OpenBLAS-0.3.9dev with GCC version 4.8.5, OpenBLAS uses 32 threads in parallel. We used the sparse QR function of MKL (version 2020.0.166) on Intel Xeon Gold 6248 CPU@3.90GHz with GCC version 5.4.0 as the performance benchmark.

GCN classifier Setup: You need to install the torch-geometric library and its four relevant libraries. For the specific process, please refer to /GCN_classifier/README.md

STM-Multifrontal QR Setup: 1. In the NUMA architecture, the libnuma library is required. The underlying relevant libraries are LAPACK and OpenBLAS. 2. Typing "sudo make relevant_lib" can install the relevant libraries that STM-Multifrontal QR needs for different reordering methods. 3. Then typing "make" can compile the default optimal STM-Multifrontal QR factorization library. 4. To verify the efficiency of our optimization strategies, you can compile the Mid_version to obtain our mid-term version library.

Please refer to README.md for the tutorial on running tests and adjusting the reordering methods for comparison.

Author-Created or Modified Artifacts:

Persistent ID:

↪ <https://zenodo.org/badge/latest/doi/355837565> ;
↪ <https://github.com/lsl036/STM-Multifrontal-QR-factorization-combined-with-GCN.git>
↪ [factorization-combined-with-GCN.git](https://github.com/lsl036/STM-Multifrontal-QR-factorization-combined-with-GCN.git)

Artifact name: STM-Multifrontal QR Factorization

↪ combined with GCN

Citation of artifact: SuiteSparse

BASELINE EXPERIMENTAL SETUP, AND MODIFICATIONS MADE FOR THE PAPER

Relevant hardware details: Intel Xeon Gold 6248 CPU@3.90GHz 20cores with TITAN RTX-24G (for GCN classifier); TaiShan 2280 V2 Server (For STM-MQR)

Operating systems and versions: Linux ubuntu 4.4.0-142-generic(X86); CentOS Linux version 4.18.0-193.1.2.el7.aarch64(ARM)

Compilers and versions: gcc version 5.4.0(X86); gcc version 4.8.5(ARM)

Applications and versions: None

Libraries and versions: MKL version2020.0.166, PyTorch 1.4.0, PyTorch-geometric 1.6.0 ; OpenBLAS-0.3.9dev

Key algorithms: Graph convolutional network; Multifrontal QR factorization

Input datasets and versions: 1864 square matrices from SuiteSparse Matrix Collection (shown in STMMQR/STM-MQR.xlsx)